

Trabalho Prático 03

Disciplina Algoritmos I

Raphaela Maria Costa e Silva - 2020006973

Departamento de Ciência da Computação – Universidade Federal de Minas Gerais

Belo Horizonte – MG – Brasil

rapha470@ufmg.com

1. Modelagem Computacional

Essa documentação tem como objetivo explicar o problema de maneira sucinta, explicar as estruturas e os algoritmos usados para solucionar o problema, analisar a complexidade de tempo do código.

A proposta do TP foi implementar um programa que retorna as dimensões da maior mesa retangular que cabe em uma planta arquitetônica. O problema foi resolvido encontrando todos os maiores retângulos que cabem na planta e depois comparando as dimensões encontradas com a dimensão de cada mesa, até achar a maior que cabe.

Para resolver o problema usei um algoritmo que maximiza o tamanho do retângulo que contem 1's para então encontrar os retângulos com maior área. E então as mesas foram ordenadas decrescentemente em relação à área, e para o critério de desempate, se caso duas ou mais mesas tivessem áreas iguais, elas foram ordenadas em relação à sua largura. Finalmente, as mesas já ordenadas são comparadas com as dimensões encontradas dos retângulos e a primeira mesa que couber é a solução.

2. Descrição da Solução

Para resolver o problema foi usado o algoritmo de achar a maior área em uma matriz de zeros e uns, em que os zeros são os lugares da planta que já está ocupado e os uns são os lugares que se pode colocar a mesa.

Vamos supor que o número de linhas na matriz dada seja 'R'. Da mesma forma, suponha que o número de colunas seja 'C'. A matriz pode ser dividida em linhas de 1 a 'R'.

Vamos pegar a primeira linha da matriz. Cada elemento da matriz com valor 1 tem $1m^2$. Da mesma forma, cada elemento da matriz com valor 0 tem $0m^2$. Assim, a primeira linha pode ser convertida para um histograma se transformamos as linhas da matriz em um único vetor.

Devemos encontrar todas as maiores áreas retangulares para resolver o problema. Para fazer isso foi necessário achar as maiores áreas para soma de cada linha da planta.

Finalmente, após ordenar as mesas, basta comparar as dimensões da mesa com as do retângulo e comparar as dimensões da mesa rotacionada com as do retângulo e a primeira mesa que couber é a solução ótima.

3. Análise de complexidade de tempo assintótica

A função para achar os maiores retângulos precisa iterar por toda a linha L , e na função `main` é preciso chamar essa função para toda coluna C que foi somada ao vetor da planta (para achar todos os retângulos). Então para encontrar os maiores retângulos, a complexidade é $O(L*C)$.

Para ordenar o vetor de mesas segundo a área, foi usado `std::sort` que é $O(M*\log M)$, em que M é o tamanho do vetor de mesas, já a ordenação para desempate foi feita usando bubble sort, que é $O(M^2)$.

Então para escolher a mesa que é a solução, o algoritmo compara cada mesa com cada retângulo, ou seja, $O(M*R)$. Em que M é o tamanho do vetor de mesas e R é o tamanho do vetor de retângulos.

Portanto, no geral sem considerar a complexidade de ler a entrada, a complexidade do algoritmo fica como $O(L*C) + O(M*\log M) + O(M^2) + O(M*R)$. Consequentemente, a solução encontrada é $O(L*C) + O(M^2) + O(M*R)$.

4. Instruções para compilação e execução

O programa foi desenvolvido na linguagem C++, compilada por G++ da GNU Compiler Collection no SO Linux.

Compile os arquivos usando o comando “**g++ tp03.cpp -o tp03**”.